Seminar workshop

Codex for Research

Let the agent do the work. Keep the judgement.

Pann Phetra, College of Sustainability and Tourism
Ritsumeikan Asia Pacific University

Install

5 minutes, free account works

Two real projects

A thesis and a data case study

Your turn

A step-by-step guide on GitHub

A chatbot answers. An agent works in your folder.

Chatbot	Agent (Codex)
Replies in a chat window	Reads and writes files in one folder
You copy results out by hand	Saves results where you work
Cannot run anything	Runs code and checks the output
Forgets what it told you	Every change can be reviewed and undone

What you need

1

A ChatGPT account

Free is enough: Codex with GPT-6 Luna, OpenAI's most efficient model

2

The ChatGPT desktop app

macOS, Windows or Linux

3

One folder

Your project, or my example data

> Guide: github.com/Pann13223029/codex-seminar

Install in five steps

1	**Download** the ChatGPT desktop app	learn.chatgpt.com/docs/quickstart
2	**Sign in** with your ChatGPT account	Free works
3	**Open a folder**	Codex only sees this folder
4	**Choose Codex**, then New chat	In the ChatGPT dropdown
5	**Check the model** below the message box	GPT-6 Luna; Light or High effort

Live demo now. Optional: install Git so you can review and undo changes.

On the Free plan: make your usage last

One task per chat

Start a New chat for each job; long chats get re-read every time

Match the effort

Light for quick jobs, High for analysis. GPT-6 Luna is already the efficient model

Plan before work

Fixing a plan is cheap; redoing finished work is not

Think it through elsewhere

Draft prompts in a regular chat; check results in a spreadsheet

On Plus or higher, stay on GPT-6 Luna for these tasks. Save Astra, Max and Ultra for hard problems.

The way of working



Brief once

AGENTS.md: the project, the data, the rules

Plan before work

"Write a plan and wait for my OK"

Check every step

Small steps are easy to check and undo

AGENTS.md: the brief Codex reads first

```
# Project brief
- Question: does "4 in 10 plants generate"
  understate how much waste is covered?

## Files
- data/incinerators_fy2005_fy2024.csv:
  one row per plant-year

## Rules
- Plan first; wait for my OK.
- Never change the original data.
- Every number must come from code you ran.
- Never invent references or statistics.
```

Write it once

Codex reads it before every task in that folder

Put your rules in it

Your standards apply every time

Template in step 02

Copy, fill in, done

A good prompt has six parts

1 Goal	What you want, and what it is for
2 Context	Which files, who reads it, what your words mean
3 Task	The steps, in order
4 Output	Format, length, where to save it
5 Checks	How you will know it is right
6 Limits	What not to do; plan first, wait for my OK

> "analyse my data" becomes six labelled lines.
> Examples to copy: prompt-guide/

Let a free chatbot write the prompt

Rough idea → **Prompt builder** in Claude, Gemini or ChatGPT → It asks you questions → **Six-part prompt** → Codex

Why

Thinking it through is the hard part. A chat is a good place for it, and it saves your Codex usage

Careful

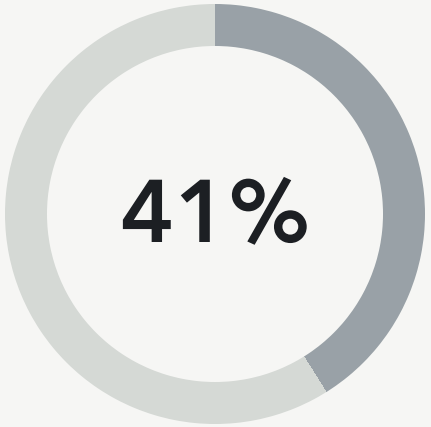
The chatbot cannot see your files: it uses [placeholders] you fill in

Never

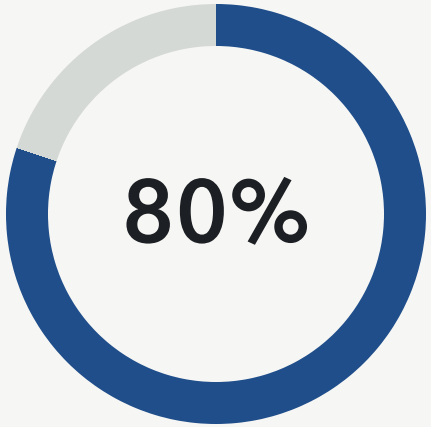
Paste private data into the chat. Describe it instead

Example 1: my graduation thesis

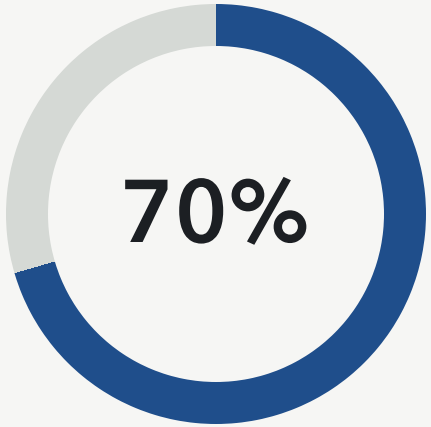
Japan says about 4 in 10 waste incinerators generate electricity. Is that low?



of plants



of waste



of capacity

20 years of Ministry spreadsheets, 23,593 records linked into 1,690 plant histories. FY2024 shown. Headline: Ministry of the Environment FY2024 summary, 415 of 991 incinerators (41.9%).

Who did what

The AI (Codex, Claude Code)	Me
Helped write the code that reads 20 spreadsheets	The question and what counts as an answer
Wrote and ran the statistics I specified	Every definition: what "generates" means
Built figures, tables and this kind of deck	Checking the numbers against the data
Drafted text from sources I had checked	What the results mean, and the final text

> Fast hands, not the brain. The thesis declares how AI was used.

Make mistakes hard to miss

Tests in the code

Plants that must link are checked every run; the result cannot change if the rows are shuffled

The build stops

If a number in the text no longer matches the results file, the thesis will not build

A mock examiner

An AI attacks the thesis, another defends it, I decide what to fix

> Ask the agent to check its own work, then check the check yourself.

Example 2: a data analytics capstone

Google Merchandise Store, one year of web analytics, 903,653 sessions in BigQuery. Which channels deserve the marketing budget?

41%

of revenue came from Google's own employees, hidden as "Referral"

98.8%

of the top 1% "likely buyers" in Google's own teaching model were Google staff

89%

of Display's credit came from one corporate buyer

Ask who is buying before you model

What worked

Asking "who is actually buying?" before modelling

Rebuilding Google's lab from its own code to test it

Checking my filter against the true source: 99.2% correct

What went wrong

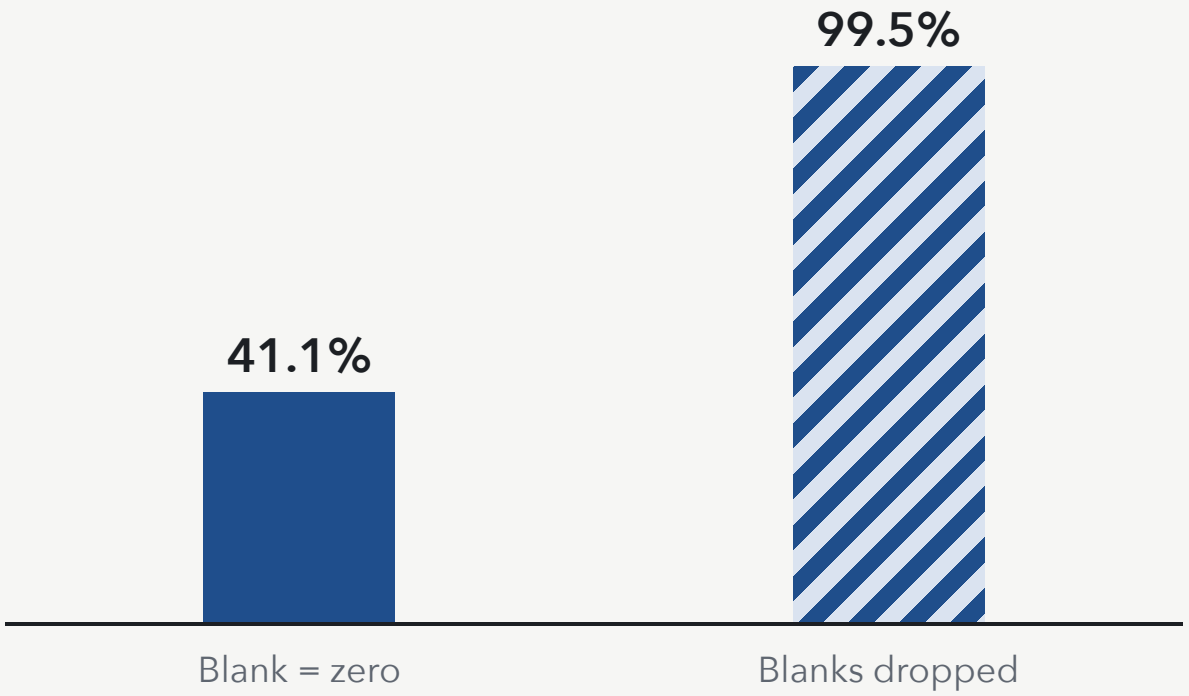
The AI stated facts about the lab from memory, twice wrongly

Caught only by checking the original source

Rule since then: verify before you present

> "Analysis designed and directed by me; code written with AI assistance."

One rule for blanks, two different answers



Share of FY2024 plants with a generator.

In 2024 the survey left 595 non-generators blank; in 2005 it wrote 0. Drop the blanks and you get 417 of 419.

Always ask: how were missing values treated?

Three example skills, ready to install



data-explore

Any spreadsheet: asks how blanks count, charts, findings, one number to check by hand

survey-summary

Forms export: removes names and emails, tables, themes with exact quotes

presentation-page

Findings to slides: outline first, one file, opens offline, saves as PDF

Install by pasting one prompt (step 04, Part A). No files to create by hand.

Make your own skill in one prompt

```
$skill-creator I want a new skill called
meeting-notes. It turns my rough meeting
notes into a five-line summary, decisions,
and a table of Task, Who and By when.
It must never invent tasks, names or dates;
mark anything unclear with [CHECK].
Use it when I ask to tidy meeting notes.
```

Create

Codex asks a few questions, then writes SKILL.md for you

Test

Type **$meeting-notes**, or just ask; the description decides

Improve

"Add a Status column to my skill." One more prompt

Slides that do not look AI-generated

editorial

Off-white, deep blue. Most seminar talks

bold

Charcoal, amber. Pitches and bright rooms

field

Grey-green, forest. Fieldwork and tourism

Do	Don't
Titles that state the takeaway	Gradients, glow, emoji bullets
One idea and one accent per slide	Three identical icon cards
Every number from your memo, with a source	"Unlock", "elevate", "delve"

Your turn: the guide

00 · 01	Install, sign in, first task
Prompts	Six parts, examples to copy, the prompt builder
04	Install the example skills, then make your own
02 · 03	Your own project: brief, plan, verify, disclose
98	**No project?** A practice survey to slides and PDF
99	**No project?** My thesis data, then your own skill

> github.com/Pann13223029/codex-seminar

Before you use any result

- ✓ Recompute two numbers by hand
- ✓ Ask how blanks were treated
- ✓ Check what each definition counts
- ✓ Check every reference exists
- ✓ Ask "what could be wrong?"
- ✓ Rewrite what you cannot explain

> Disclose it. Undisclosed AI help can count as plagiarism. Check your course rules; template in step 03.

Four ways to get it wrong

Private data in the folder

Interviews, personal data, passwords: keep them out

Invented references

They look real. Search for every one

Trusting a confident answer

Ask for the source and the file behind each number

Handing over the thinking

The question and the meaning are yours

If you remember one thing

Let the agent do the work. Keep the judgement, and check before you trust.

The guide

github.com/Pann13223029/codex-seminar

Questions

Thank you